# IMD3002 – 3D COMPUTER GRAPHICS

Prof. Chris Joslin

## L22 – Rendering: NPR + Rotoscoping

# Overview

- This lecture covers the topics of non-photorealistic rendering and network rendering

# Non-Photorealistic Rendering

# Non-Photorealistic Rendering

"Using a term like 'nonlinear science' is like referring to the bulk of zoology as 'the study of nonelephant animals" – Stanislaw Ulam

# Non-Photorealistic Rendering

- Introduction
  - We have seen over many slides that one of the main aims of the industry is to render everything as (photo) realistically
  - Everything from GI to the use of caustics shows how we want everything to look real, or as close to real that no-one cares
  - This is useful for more immersive and cool looks in Virtual Environments and Games, entertaining and believable 3D animations, and helping in architectural renderings

# Non-Photorealistic Rendering

- Introduction
  - Sometimes, we don't want to have this realism and we actually want to have something which is actually "non-photorealistic"
  - In this case we call upon a Non-Photorealistic Renderer (NPR) and in it has many different purposes
  - This technique, sometimes called "Stylistic Rendering"
  - Interesting enough, NPR can be actually harder to achieve, because it is more about style than maths and physics …

# Non-Photorealistic Rendering

- Introduction
  - Only those details that are relevant to the goal of the particular application are the ones that will be displayed using NPR
    - **Example:** a photograph of a nice shiny Aston Martin may be useful for selling the car to a customer, but in order to repair the engine a simplified line drawing with the relevant parts highlighted may be more meaningful

# Non-Photorealistic Rendering

- Introduction
  - Another area of NPR is in the simulation of painterly styles and natural media (e.g. pen, ink, charcoal, watercolours, etc)
  - It is a huge field that lends itself to an equally large variety of algorithms that attempt to capture the feel of various media

# Non-Photorealistic Rendering

- Introduction
  - There are several techniques, and thus several styles that are available, as the name suggests there are many methods – here are some of the most popular:
    - Toon Shading
    - Silhouette Edge Rendering
      - Surface Angle Silhouetting
      - Procedural Geometry Silhouetting
      - Silhouetting by Image Processing
      - Silhouette Edge Detection

# Non-Photorealistic Rendering

- Toon Shading
  - There has been a large amount of attention given to one particular area of NPR, that of Cel or Toon Rendering
  - This style is identified with cartoons
  - At its simplest objects are drawn using solid lines separating different areas of solid colour
  - One reason it is popular is because of what McCloud* describes as "amplification through simplification"

*Understanding Comics ~ Scott McCloud

# Non-Photorealistic Rendering

- Toon Shading
  - In this case, by cutting out the clutter we can amplify the effect of information relevant to the presentation – for cartoon characters, a wider audience will identify with those drawn in a simpler style
  - We also see this technique in games such as *Cel Damage*, and *Jet Grind Radio*

Cel Damage

# Non-Photorealistic Rendering

- Toon Shading
  - There are a number of different techniques that can be applied for toon rendering
  - Models with textures, but no lighting, a solid-fill cartoon style can be approximated by quantising textures (i.e. dividing out the pixel values to simpler values)
  - For shading, the two most popular methods are to fill the polygons with solid colour (i.e. unlit) or use a two-tone approach (lit and shadowed areas)

# Non-Photorealistic Rendering

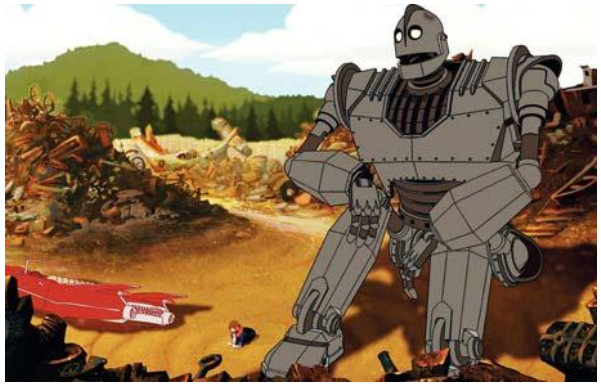
- Toon Shading
  - Solid Shading is trivial, and the two-tone approach (sometimes called hard-shading) can be performed by remapping the traditional lighting equations to different colour elements
  - Silhouettes are often rendered explicitly in a black colour, which amplifies the cartoon look
  - Certain techniques employ a diffuse shading dot product for each vertex and use this value as a texture coordinate to a one-dimensional texture map (effectively a colour gradient)

# Non-Photorealistic Rendering

- Toon Shading
  - The gradient itself contains only a few colours (often only 2) and the implementation can be done using vertex shaders
  - The standard use of the diffuse term is that $n \cdot l < 0$ means that the surface is facing away from the light and therefore it is in shadow (however we can remap it to other values as desired)
    - Where n is the normal vector, l is the light vector
    - If $n \cdot l < 0$ then the light vector is facing away towards the opposite side of the surface (and thus unlit)

# Non-Photorealistic Rendering

- Toon Shading
  - Several cartoon series have made use of NPR extensively and as the technique becomes popular more will undoubtedly follow:



Iron Giant

Simpsons Movie

Futurama

# Non-Photorealistic Rendering

- Silhouette Edge Rendering
  - Algorithms for Cel Shading reflect some of the major techniques in NPR
  - Methods used can be categorised as based on surface angle, image processing, procedural geometry, vector edge detection, or a hybrid of these techniques

# Non-Photorealistic Rendering

- Silhouette Edge Rendering
  - There are a number of different types of edges that can be used in toon rendering:
    - A **boundary** or **border edge** ~ one that is not shared by two polygons (e.g. a paper edge), a solid object typically has no boundary edges
    - A **crease** or a **hard edge** ~ one that is shared by two polygons, but the angle between them is greater than some defined value (e.g. 60°) – alternatively we can look at the vertex normals (e.g. a cube has crease edges)

# Non-Photorealistic Rendering

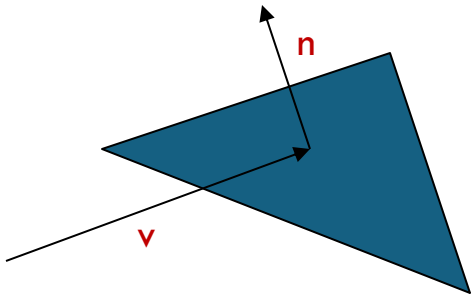
- Silhouette Edge Rendering
  - Toon Edge types (Cont'd):
    - A **material edge** ~ which appears when two polygons sharing an edge differ in material – this can also be an edge that an artist wants to display (e.g. forehead lines or a line to separate the same coloured pants and shirt)
    - A **silhouette edge** ~ in which two neighbouring polygons face in different directions compared to some direction vector
  - This categorisation is based on common usage, but there are some variations (and mixing)

# Non-Photorealistic Rendering

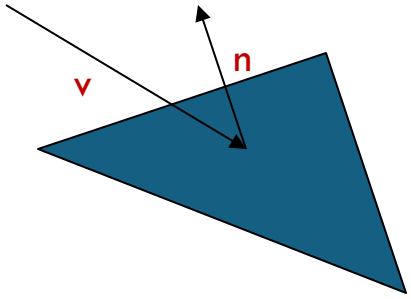
- Silhouette Edge Rendering
  - For toon rendering the direction used to define silhouette edges is the vector from the eye to some point on the edge
  - Silhouette edges are therefore those for which one neighbouring polygon is front-facing and the other is back-facing

# Non-Photorealistic Rendering

- Surface Angle Silhouetting
  - The dot product between the direction to the viewpoint (v) and the surface normal (n) can be used to provide a silhouette edge
  - If this value is near zero, then the surface is nearly edge on to the eye and so is therefore likely to be near a silhouette edge



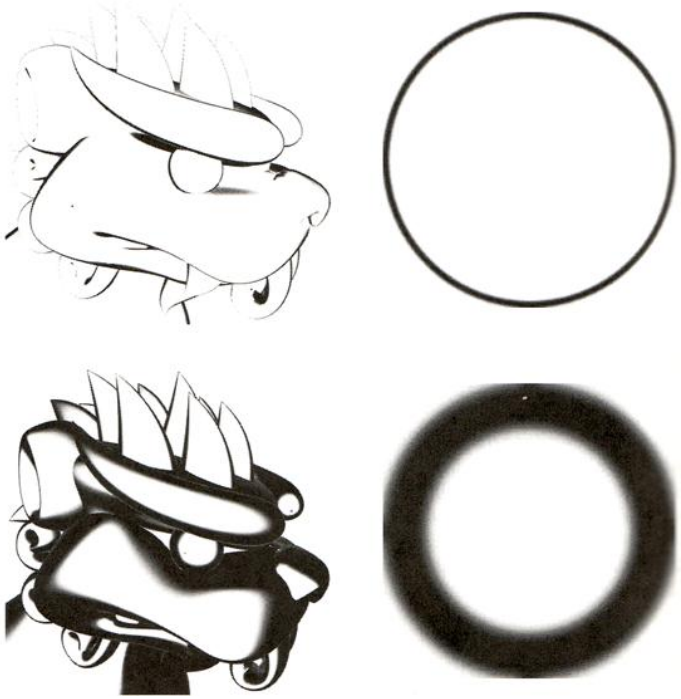

Triangle near edge




Triangle not near edge

# Non-Photorealistic Rendering

- Surface Angle Silhouetting
  - The technique is equivalent to shading the surface using a spherical environment map with a black ring around the edge



# Non-Photorealistic Rendering

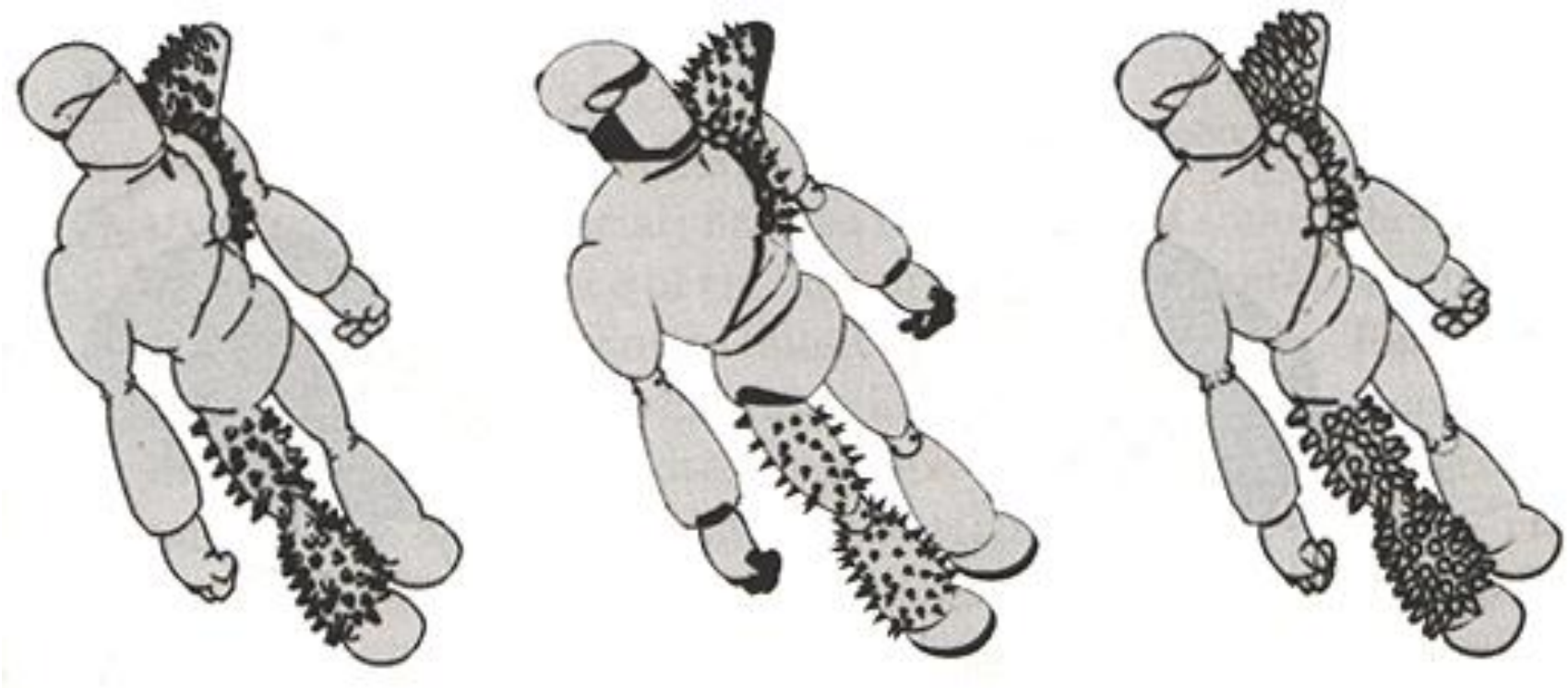
- Procedural Geometry Silhouetting
  - The basic idea is to render the front-faces normally, then render the back-faces in a way to make their silhouette edges visible
  - There are a number of methods that can be used to achieve this
  - All techniques first draw the front-faces, and then front-face culling is turned on and the back-face culling turned off so that only the back-faces are displayed

# Non-Photorealistic Rendering

- Procedural Geometry Silhouetting
  - One method to render the silhouette edges is to draw only the edges (not the faces) of the back-faces
  - In this way, all lines except the silhouette edges are hidden – this technique works well for single pixel wide lines) ~ however in OpenGL lines can be wider than this (not in DirectX) there is nothing joining the separate lines together, resulting in noticeable gaps

# Non-Photorealistic Rendering

- Procedural Geometry Silhouetting
  - **Example:** back-facing edge drawing with thick lines

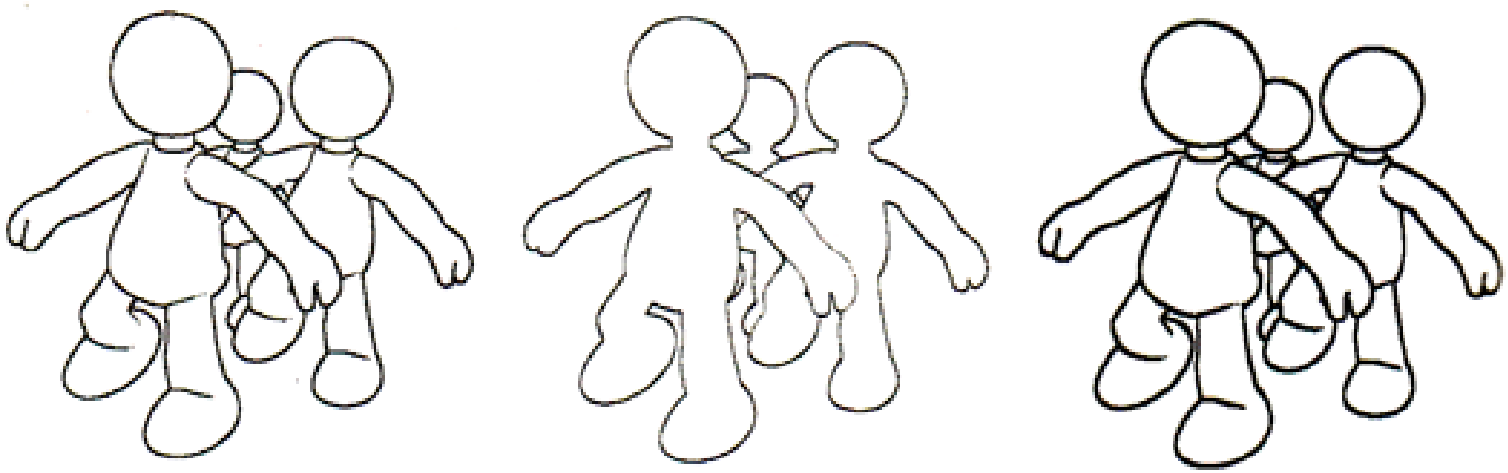


# Non-Photorealistic Rendering

- Silhouette by Image Processing
  - A technique that is more image-based, due to the fact that it works only on the data stored in the buffers (and not the geometry) is called Silhouetting by Image Processing
  - The idea is fairly simple, by looking for discontinuities in neighbouring z-buffer values we can find most of the silhouette edges
  - Discontinuities in neighbouring surface normals indicate boundary edges
  - Finally rendering the scene in ambient colours detects the edges the other techniques miss

# Non-Photorealistic Rendering

- Silhouette by Image Processing
  - **Example:** here is an example of edges found by processing the normal and z-depth map of the scene; the image on the right is the thickened composite of the two images (note the normal map was actually sufficient to detect all edges)



# Non-Photorealistic Rendering

- Silhouette Edge Detection
  - The previously described techniques have the disadvantage that they require a multi-pass rendering to render the silhouette
  - The second pass for the procedural geometry methods typically test more pixels than are actually shaded
  - Also, there are also various problems with thicker edges and little control over the style that is actually rendered

# Non-Photorealistic Rendering

- Silhouette Edge Detection
  - Image-based methods have similar problems with creating thicker lines
  - Another method is to detect the edges and render them directly
  - This form of silhouette edge rendering permits a finer control over how the lines are actually rendered; another advantage is that since the edges are independent of the model it is possible to create effects (e.g. have the silhouette jump, whilst the mesh is frozen “in shock”)

# Non-Photorealistic Rendering

- Silhouette Edge Detection
  - The silhouette edge is one where one of the two neighbouring polygons faces toward the viewer and the other faces away
  - The test is based on the following equation:

$$\left(n_0 \cdot v > 0\right) \neq \left(n_1 \cdot v > 0\right)$$

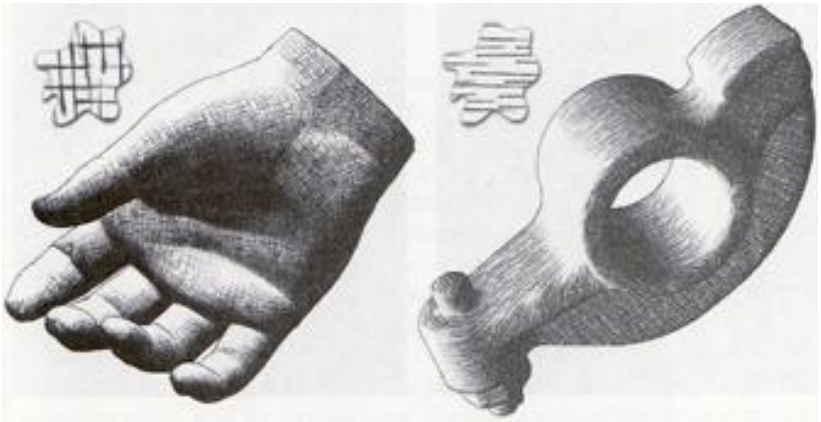
  - When $n_0$ and $n_1$ are the two surface normal vectors and $v$ is the view direction to the edge
  - For this test to work, the surface must be consistently oriented

# Non-Photorealistic Rendering

- Other Styles
  - Although toon rendering is a popular style, there are various other styles
  - NPR effects can range from modifying realistic textures, to having the algorithm procedurally generate geometric embellishments from frame to frame

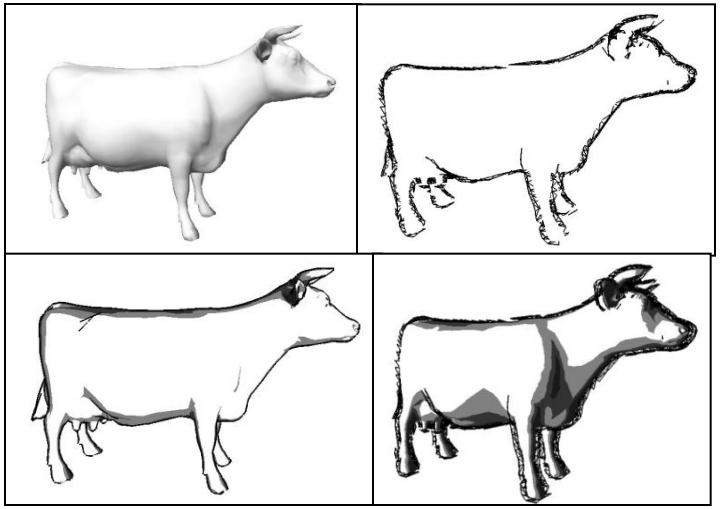
# Non-Photorealistic Rendering

- Other Styles
  - One of the most popular techniques is called "Tonal Art Maps" or TAMs



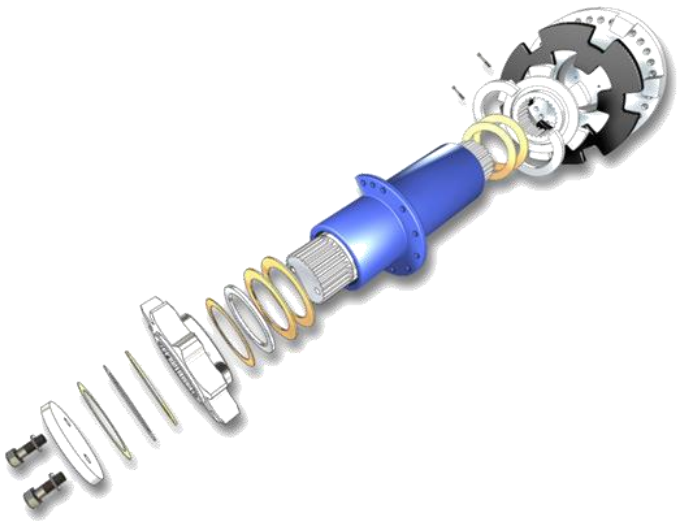
# Non-Photorealistic Rendering

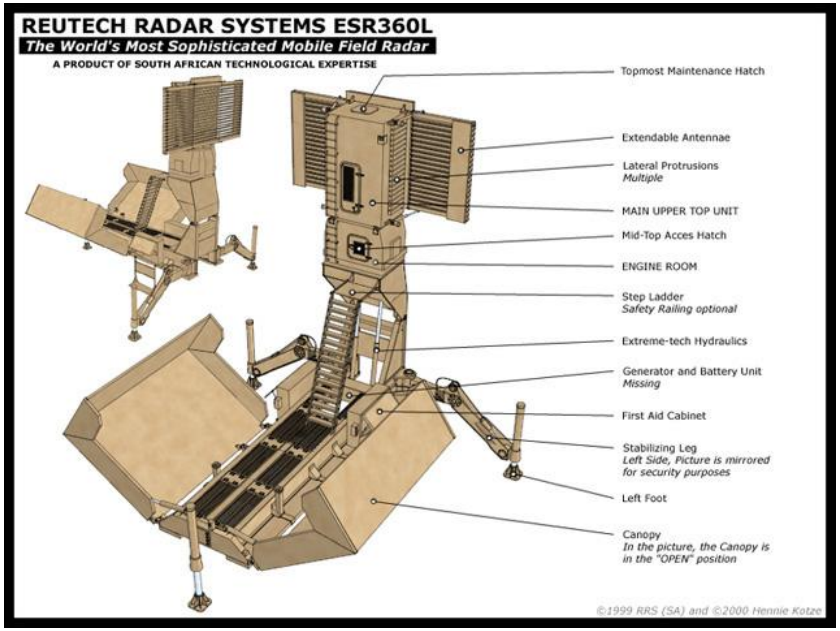
- Other Styles
  - **Painting** ~ another type of NPR is painting, this tries to make everything as if it were painted by hand
  - **Sketch** ~ pretty much the same goes for sketching (as shown above)



# Non-Photorealistic Rendering

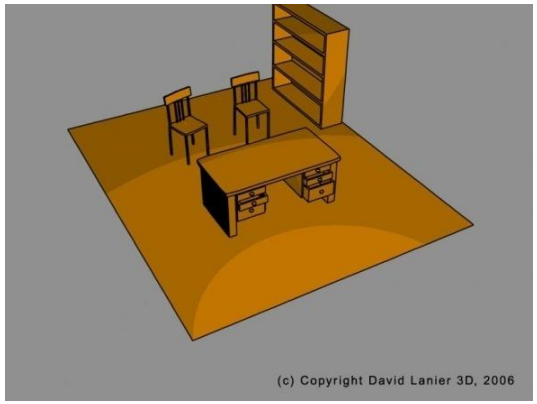
- Other Styles
  - **Schematics + Exploded Views** ~ another type of NPR which can be very useful is exploded views (of complex components) and schematic diagrams

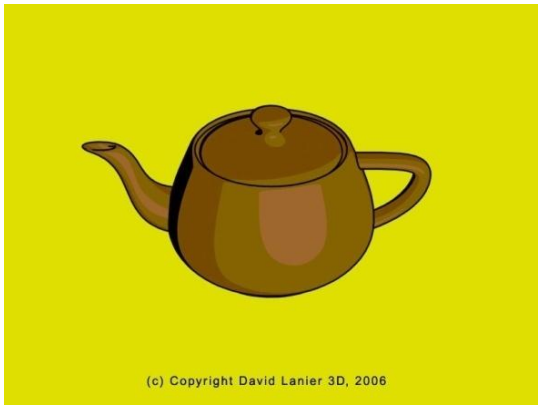




# Non-Photorealistic Rendering

- Applications & Plug-ins
  - As previously discussed, NPR is more specialised compared to a standard GI renderer – therefore we can look at different applications and plug-ins available for 3D
    - **Maya Toon** ~ Maya has its own built in toon shader – which basically encompass different material "interpretations" and paint effects



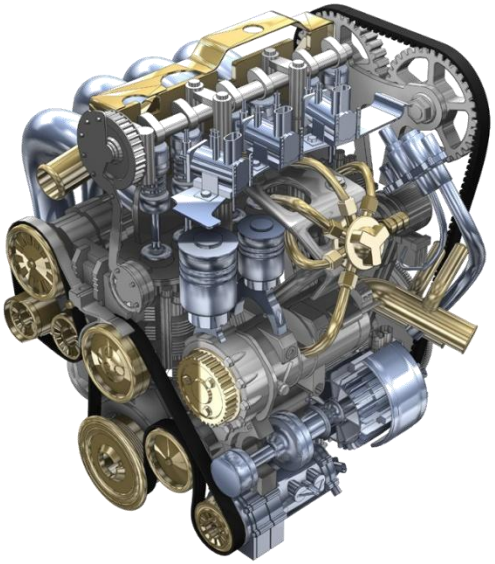



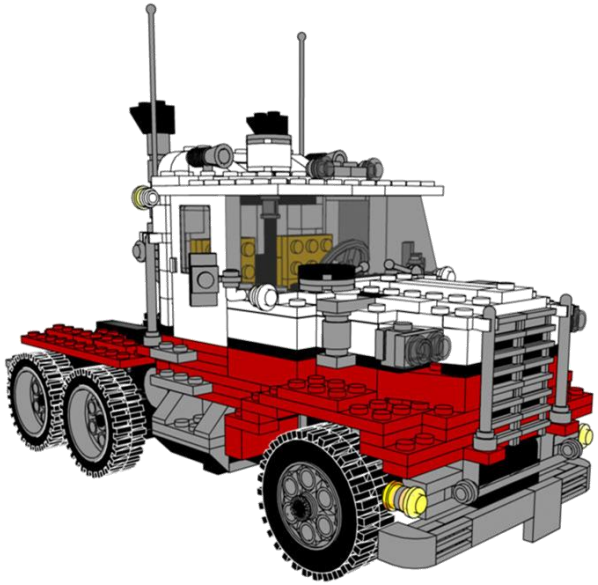
# Non-Photorealistic Rendering

- Applications & Plug-ins
  - **CartoonReyes** ~ this is one of the Reyes plugins (www.reyes-infografica.com) that can be used for cartoon-like shading (available for 3DSMax only)

# Non-Photorealistic Rendering

- Applications & Plug-ins
  - ***FinalToon*** ~ available from cebas (www.cebas.com - who also make FinalRender) – works only on 3DSMax, but has some very prominent example renderings





# Non-Photorealistic Rendering

- Applications & Plug-ins
  - **Illustrate!** ~ this package has been used on some very classic movies such as "Ghost in the Shell" and "Corto Maltese" – works only on 3DSMax currently (although Maya version is apparently coming)
    - http://www.davidgould.com/Illustrate/

# Non-Photorealistic Rendering

- NPR Quake
  - One of the more interesting real-time applications of NPR is in NPR-Quake
  - It utilises something called the “Sketchy-Renderer” which essentially “attempts to simulate a person rapidly sketching the Quake environment on a piece of white paper
  - The walls are shaded with a hash-style sketching, and the water ripples with every stroke of the writing utensil”

# Non-Photorealistic Rendering

- NPR Quake
  - The source files can be found here
    - http://www.cs.wisc.edu/graphics/Gallery/NPRQuake
  - Some screen shots …

http://www.youtube.com/watch?v=C78PDSRbpC8

# Rotoscoping

- Introduction
  - In order to be complete, we should have a quick look at Rotoscoping
  - Several methods are said to be based on Rotoscoping, but generally are not – these include optical motion capture and non-photorealistic rendering
  - Essentially, Rotoscoping is the technique in which animators sketch over a live-action film, frame-by-frame

# Rotoscoping

- Introduction
  - It is used for a variety of purposes – for example to add the lightsaber effects to the original "Star Wars" films, and more recently in the film "A Scanner Darkly (2006)"

# Rotoscoping

- Technique
  - The technique is mainly performed by a skilled animator, but the term has come to be used for techniques that produce mattes (object silhouettes) in order that scenes can be layered
  - Therefore, the term often pops up when discussing blue/green chromakey work

# Network + Distributed Rendering

# Network + Distributed Rendering

- Introduction
  - As previously discussed rendering can take a long time, especially if you are using GI and attempting to obtain really realistic results
  - Even the most powerful of today's computers are not able to render a feature-film style film in a few seconds, let alone in real-time
  - It is therefore important to use more than one computer to perform this task

# Network + Distributed Rendering

- Introduction
  - We could of course start a rendering on many different machines, but this is cumbersome and tedious – more than anything we would have to start a lot of render jobs on each machine, but also we would have to take care if this render failed or completed on ahead of time (the rendering of frames is not equal)
  - Not only that, but we would have to make sure that those machines were free – if we use a workstation which is designed for a human we have some wasted resources

# Network + Distributed Rendering

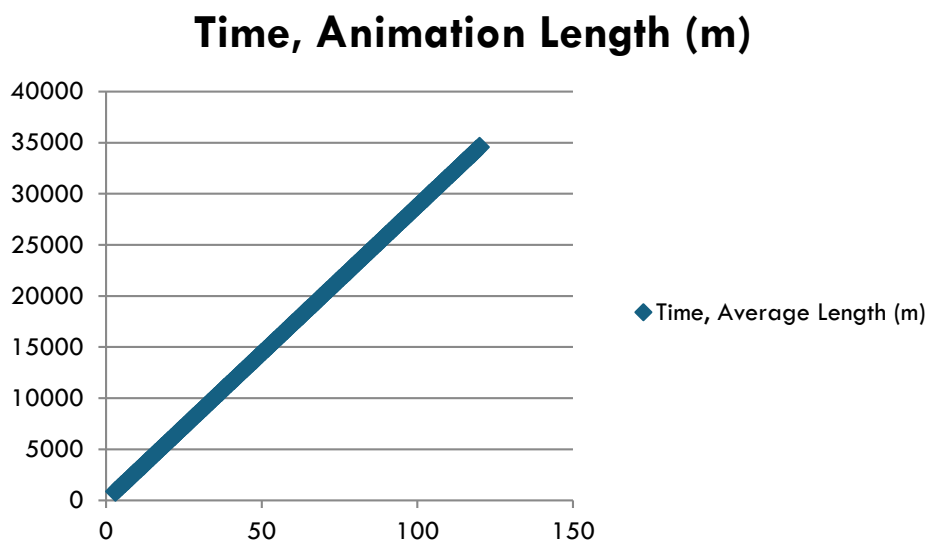
- Rendering Time Equation

$$T_{TRT} = T_{APF} \times C_{RR} \times T_{AL} \times FPS$$

  - $T_{TRT}$ = Time, Total Render Time (s)
  - $T_{APF}$ = Time, Average Per Frame (s)
  - $C_{RR}$ = Count, Re-Renders Per Frame
  - $T_{AL}$ = Time, Animation Length (s)
  - $FPS$ = Frames Per Second

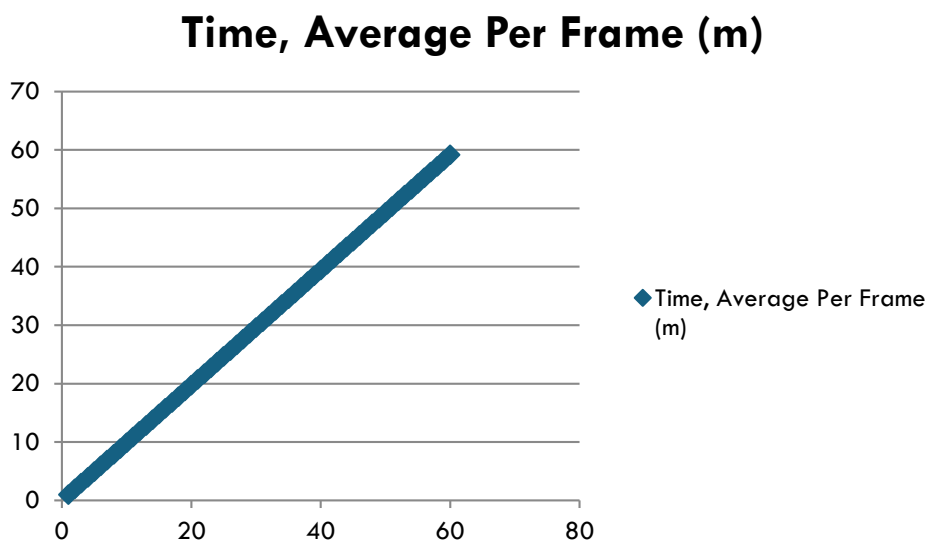
# Network + Distributed Rendering

- Time, Animation Length
  - $C_{RR} = 4$
  - $FPS = 24$
  - Minutes / Frame = 4
  - Results in Hours



# Network + Distributed Rendering

- Time, Animation Length
  - $C_{RR} = 4$
  - $FPS = 24$
  - Total Animation Length = 1Hr 30m
  - Results in Years



# Network + Distributed Rendering

- Introduction
  - Therefore, we can use either "**Network Rendering**" or "**Distributed Rendering**" as a package to handle large render jobs
  - In this section we shall examine some of considerations when using either of these two types
  - First, let's look at how they both work ….

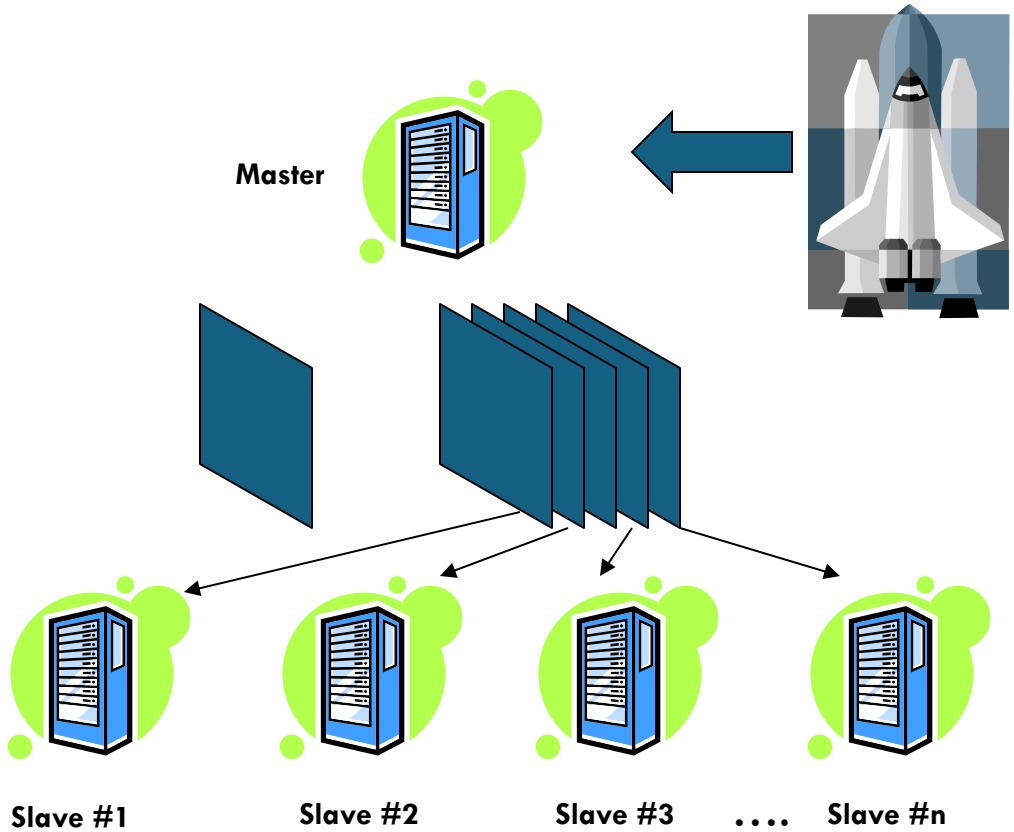
# Network Rendering

- Introduction
  - Network Rendering works on the basis of a scene being broken down into frames and each frame being sent (via the network) to a different machine (often referred to as a node) to be rendered
  - When the machine has rendered the frame, it is place in a specific directory and the machine waits for the next frame
  - This group of machines is called a "**Farm**"

# Network Rendering

- Introduction
  - **Example:** diagram exemplifying this process

The entire scene is passed to the master, and then each frame is passed to different slave machines



# Network Rendering

- Introduction
  - Network Rendering is generally started by a workstation on a client, which is where the scene has been created (or at least loaded)
  - However, we often use another machine to actually manage the distribution of each of the frames – this is referred to as a "**Master**", "**Server**", or "**Manager**" depending on the system
  - This is in contrast to the machines that actually do the work called "**Slaves**", "**Clients**", or "**Satellites**"

# Network Rendering

- Introduction
  - It is usual for the "**Master**" to also perform rendering and have a "**slave**" running at the same time – although this depends on how big the farm is: the more nodes in the farm, the more work the Master has to do just making sure everything is working
  - The "Master" has two jobs, the first (and foremost) is to distribute all relevant information to the "Slaves" in the farm
  - The second is to resubmit a job if a slave fails to render the frame

# Network Rendering

- Introduction
  - Failure is rare, but not totally absent, for large farms (small ones too); this can be for several reasons:
    - **Transmission Error** ~ each node relies heavily on the data it is presented, it has no verification method, therefore if there is an error in transmission (i.e. a read from disk, etc) then an error on that node can occur
    - **Computational Error** ~ GI is complex and involves many calculations; it is rare, but it can exist that the calculations for a particular frame accumulate enough errors for the frame, or the node, to crash

# Network Rendering

- Storage + Maps
  - In many respects, it is important to consider Network Rendering early on in the development stage – this is because rendering on a network is very different to rendering on a single workstation
  - There are several considerations for this in terms of time/capacity etc, but two of the main issues that we must consider first are Storage and Maps
  - We shall discuss Maps first

# Network Rendering

- Maps
  - We have discussed briefly textures and texturing – in most 3D modelling applications links to these textures are stored in some kind of catalogue so that as we model we can apply the same texture to different models
  - In general, these textures (referred to as Maps) can be all over the place (different directories, disks etc)
  - The problem arises when it becomes time to render with a farm – this because each node should be considered totally separate

# Network Rendering

- Maps
  - This means that whatever paths and shortcuts that the modeller has created, probably won't be available to the farm (which is basically independent)
  - Therefore, depending on the rendering system used (although this problem is common for most) we must have a directory for maps that is generally accessible to both the modeller and the farm (although this doesn't have to be immediate – but planning is good)

# Network Rendering

- Maps
  - In general, the farm will require a URI stem that can be seen via a network path (CIMS, NetBIOS, Samba, etc)
    - **Example:** \\MyFarm\Repository\cjoslin\CMe.mb
  - This type of URI stem can be seen by all machines on the network as it is an absolute path
    - **Example:** H:\CMe.mb
  - This might be in the same place, but H: is a mapped drive relative only to the user (modeller)

# Network Rendering

- Maps
  - Relative paths are also acceptable (and in some respects preferable); this is because they are very workable for wherever the modeller resides (and can be moved)
  - Therefore all maps are relative to the scene file
    - **Example:** ".\Maps\JuniperBush.jpg"
  - Organisation is important, but organisation based on a single directory is preferable; so instead of multiple directories everywhere, a single directory with multiple sub-directories is the optimum solution

# Network Rendering

- Storage
  - First we need to consider size, when we render we generally render a raw frame (such as a TIFF or TARGA) which can be used for putting together your video sequence
  - We render a raw, uncompressed file, for two reasons
    - Firstly, compression takes up valuable render time, which can be performed later
    - Secondly, we don't want to spend so long rendering and then have poor files because of compression artefacts – start with something good

# Network Rendering

- Storage
  - Let's take an upper limit and assume we are rendering for Blu-Ray – therefore each frame is going to be 1080 x 1920 pixels, with 3 channels (possibly 4, i.e. Alpha) in RGB – or 4 channels in CYMK …
  - We assume 3, and therefore each frame will be 6,220,800 Bytes, or just under 6 Million Bytes
  - At 60 frames per second, we can assume 373,248,000 Bytes / second of film

# Network Rendering

- Storage
  - From this figure we can derive two interesting items
    - $373 \times 10^6$ Bytes per second, means that for an average animated film (say 1Hr30), we would require 5400 seconds of video, which equates to (5400 x $373 \times 10^6$ bytes) 2,015,539,200,000 or 2TBytes worth of storage ($2 \times 10^{12}$ bytes)
    - Secondly, there is a lower limit on the time per frame – if each frame is rendered too fast, we shall have limits imposed by the hard drives and the network

# Network Rendering

- Storage
  - With regards to this second point, we should consider that currently the fastest hard drive can transfer (on average) around 83MBytes/second – so in terms of a single disk (which is unlikely) each frame would have to take longer than 5 seconds to render otherwise the disk would not be able to cope with the data rate

# Network Rendering

- Storage
  - This is an unlikely occurrence, most GI frames for an animated feature film would take at least 4-5 minutes minimum – but it is used to exemplify the bottlenecks in a farm of this sort
  - Secondly, the network capacity would override this long before: 373 x $10^6$ bytes is nearly 3GBits/second … which is well outside the capacity of a normal gigabit Ethernet network
  - In this case most connections used will be either **Fibre Channel** (4 – 10GBits/s) or **Myrinet** (2 – 10GBits/s)

# Network Rendering

- Complexity
  - There are several factors that influence the render time; whilst there are no hard rules, it is good to remember the following:
    - **Number of Polygons** (tessellated or otherwise)
    - **Number of Lights** (each light means more rays)
    - **Screen Size** (each pixel in the viewport means another ray to trace)
    - **Map Sizes** (the larger the number of pixels in a texture, the more time it takes to calculate)
    - **CPU/Memory Capacity** (faster CPU is obvious, but if we run out of memory, the system will start swapping to disk, which is terribly slow …)

# Network Rendering

- Render Time
  - Going back to the frame sizes, etc – we can also calculate the time to render
  - Most GI frames start at around 5 to 10 minutes per frame to render (depending on scene complexity)
  - Therefore, if we were to render a 20 minute movie, we would need actually be rendering 72,000 frames (at 60fps)
  - If each frame takes 10 minutes, then we would need 12,000Hrs to render all the frames (500 days)

# Network Rendering

- Render Time
  - On the school's render farm (32 usable cores), at maximum throughput, we could reduce this figure to just over 15 days
  - However, that does require that the render farm is working 24/7 for 2 weeks – no glitches, no problems with each frame, etc
  - Therefore, we have to take serious consideration for the render time, even though each frame is not very long

# Network Rendering

- Render Time
  - We can reduce the overall time by reducing the frame rate (60fps → 24fps) although this can introduce flickering and unwanted artefacts so we have to be careful
  - We can reduce the overall length of our feature, but this might reduce the overall artistic quality of the animation
  - Therefore, in all cases we need to carefully plan for rendering even if we do have a render farm at our disposal

# Network Rendering

- Limitations
  - Network Rendering has its limitations, obviously if each frame is rendered too quickly the render farm will bottleneck somewhere – generally it will not be worth it to even start the job
  - This is because there is still some overhead with rendering on the farm (setup, copying data – network access is slower than disk access, etc)

# Network Rendering

- Limitations
  - Secondly, some scenes we cannot render when there is a dynamic calculation involved
  - This means that there is inter-frame dependency and thus one frame cannot be rendered until the previous frame has been completed
  - Calculations of this kind might be derived from hair, fluid, or cloth simulations which do not know where the next position is going to be until it has been calculated

# Distributed Rendering

- Introduction
  - Distributed rendering (DR) shares certain characteristics (and problems) with Network Rendering, but is very different in others
  - DR works on the basis of each frame from a scene being broken down into sections (called Buckets) and each bucket being sent (via the network) to a different machine to be rendered
  - When that frame has been completed, the machines start on the next frame

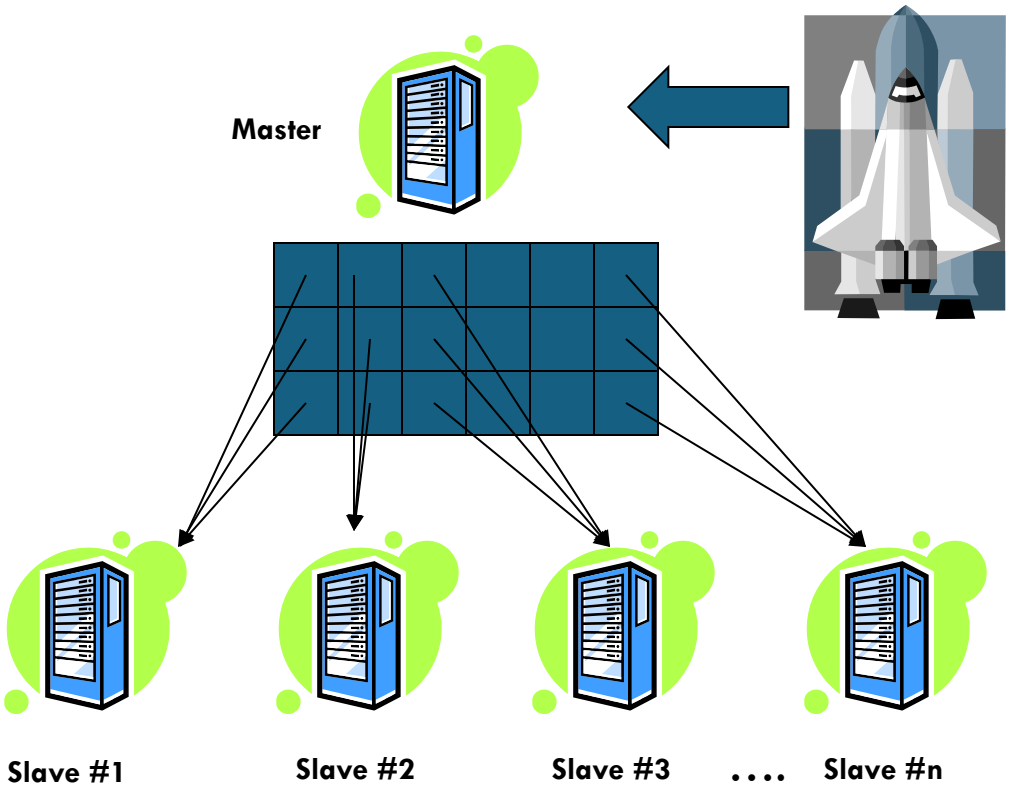
# Distributed Rendering

- Introduction
  - This is very different to Network Rendering because we are getting all machines to work on one frame at the same time
  - This changes the considerations for network and storage, because we have a massive “pulse” of data all at once – in this respect this limits the number of machines that can work on a frame at one time (although sometimes this is a physical limit, not a software limit)

# Distributed Rendering

- Introduction
  - **Example:** here is a diagram showing the general layout of a distributed rendering farm

The entire scene is passed to the master, and then each frame is split between all the available slaves (the next frame isn't started until the previous one is finished)



# Distributed Rendering

- Disadvantages
  - The disadvantages of DR are as follows:
    - **Limited number of machines** ~ as we are rendering a frame on many machines, we can in fact have a higher data rate (easily approach 373MBytes/second of real-time)
      - Even though it's a smaller part of the frame, the entire scene is still send to each machine
    - **Larger overhead** ~ in NR the overhead is per frame, so we spend time to sort the frames and distribute them to each slave; in DR we separate this into buckets and therefore per frame we have an overhead accumulated per bucket
      - This is from the network protocol and the render farm communication requirements in general

# Distributed Rendering

- Disadvantages
  - *Disadvantages cont'd*
    - **Bigger Error Margin** ~ if a frame fails in NR, it can be simply re-rendered; however, if a bucket fails there is either an error in the frame, or it has to be re-rendered (which can result in a higher overhead)

# Distributed Rendering

- Advantages
  - DR is fairly new, but it is useful (otherwise it wouldn't be used)
    - **Inter-Frame Dependency** ~ As we are rendering 1 frame at a time, we can actually avoid the problems of inter-frame dependency because we are only rendering 1 frame at a time
    - **Faster Frames** ~ sometimes we can only see when something has gone wrong once we have rendered a frame in GI – and those frames can take a long time; using DR we can see that frame much quicker and therefore reduce bugs quicker

# Network + Distributed Rendering

- Conclusion
  - It should be noted that setting up any rendering of this type is not simply a “plug-n-play” step
  - Many considerations need to be taken into account, from maps, to capacity, to render time which should be taken into account right from the start (especially the storage and location of maps)
  - GI especially requires “tweaks” to get it to work exactly as you require, pushing this onto machines out of your control makes this even harder